UNIVERSITY WEST

Department of Engineering Science

Crane Simulation Project

You build one system and extend it for each grade level. Each grade level requires the project details in the previous grade level to also be completed, so the levels are cumulative: read from Grade E and stop at the level you are aiming for.

The project should be submitted on Canvas and presented in class.

You will work with a simulation available on Canvas, shown in the following figure.



The `plug_and_produce_simulation.py`, can be downloaded from Canvas. Run it with Python. It opens the cell and serves Modbus TCP on **127.0.0.1:502**. On Linux, port 502 is not open to normal users, so the simulation uses **5020** instead and shows the port in its window — connect with `ModbusTcpClient('127.0.0.1', port=5020)`.

The simulation window has a 3D view, a **Front** and a **Top** view, the signal panel on the right, counters for delivered, scrapped and collided parts, and an event log that records every Modbus write. Use the event log when something does not behave as you expect.

A process runs for about 4 seconds and the crane moves about 250 units per second, so a move across the cell takes a few seconds. **Generate** puts a part at a source, **Reset** clears the whole cell, and you can click an output in the signal list to write it by hand.

Modbus

The simulation is controlled over the industrial protocol **Modbus**.

Importing pymodbus:

```
from pymodbus.client import ModbusTcpClient
from pymodbus.exceptions import ModbusException
import time
```

Connect to the Modbus client:

```
client = ModbusTcpClient('127.0.0.1')
```

You will interact with various resources in the simulation, each identified by specific Modbus addresses. Create functions to read from and write to Modbus registers. For example, the following function can read inputs from a certain address:

```
def read_input(address):
    result = client.read_holding_registers(address=address, count=1)
    return result.registers[0]
```

The following function can write to an address with a certain value:

```
def write_output(address, value):
    result = client.write_register(address, value)
    print(f"Successfully wrote {value} to address {address}")
```

In the following table all addresses are listed. Boolean values are `0 = False` and `1 = True`.

Resource	Signal (documentation)	Name in the simulation panel	Address	I/O
Cell	reset	reset	0	Output
Crane	setX	setX	1	Output
Crane	setY	setY	2	Output
Crane	vacuum	vacuum	3	Output
Process 1	run	startp1	4	Output
Process 2	run	startp2	5	Output
Crane	posX	atX	15	Input
Crane	posY	atY	16	Input
Source 1	sensor	source1	17	Input
Source 2	sensor	source2	18	Input
Process 1	isRunning	p1running	19	Input
Process 2	isRunning	p2running	20	Input
Process 1	sensor	p1sensor	21	Input
Process 2	sensor	p2sensor	22	Input
Panel	–	button1	23	Input
Panel	–	button2	24	Input

Notes on the signals:

- Writing to an input address does nothing — the inputs belong to the simulation.
- `reset` (0) stops both processes and clears itself back to 0 after the write.
- `startp1` / `startp2` start a process. The signal clears itself to 0 when the process has finished; writing 0 while it runs stops the process.
- `button1` and `button2` are the two buttons at the bottom left of the window. They stay 1 for at least half a second after a click.

Modbus client connect and close

You must connect Modbus using `client.connect()` and close the connection when you are done with the simulation control using `client.close()`.

Locations

Picking parts is done at `Y=82`, and the following `X` locations: Source1: `X=55`, Source2: `X=158`, Process1: `X=450`, Process2: `X=650`, Sink: `X=945`

Note that you will get collisions unless you add an offset location in Y, before and after picking or placing a part. Try to find a suitable offset.

Grade E

The crane should pick one part at source 1 and place it in process 1, then move it to the sink. You are not required to run the process for this grade. Create a JSON file with all the locations `X,Y` and instructions for opening and closing the vacuum. Write code to iterate over data from the JSON file and find the `setX` and `setY` together with the vacuum state for each set of actions, and run the crane based on the JSON file values. For picking up a part at source 1, a JSON file can look like this:

```
{
 "actions": [
  {
   "vacuum": 0
  },
  {
   "setX": 55,
   "setY": 200
  },
  {
   "setX": 55,
   "setY": 82
  },
  {
   "vacuum": 1
  },
  {
   "setX": 55,
   "setY": 200
  }
 ]
}
```

Grade D

Requires grade E. The crane only picks a part in the source if the sensor shows that there is a part there. Thus, the crane must wait for a part to be added. The process 1 should run when a part has been placed there. Thus, you must wait for the process to complete before moving the part. You must use the signal from the simulation to detect if the process has completed.

New sequence:

1. Pick one part at source 1
2. Place part in process 1
3. Run process 1
4. Move part to the sink

Make it possible to generate a part at source 1 (type 1) when a part is already at process 1 or handled by the crane. Your system must run without collisions.

Grade C

Requires grade D. Add a log entry every time a part is located in a new location. Save logs to a comma-separated file (CSV). The log entries consist of a product ID, timestamp, position data (`X,Y`), and vacuum state. The output in csv file can for example look like this:

```
product_id, timestamp, x, y, vacuum
1, 2025-10-15T16:10:49, 55, 82, 1
1, 2025-10-15T16:10:50, 55, 200, 1
1, 2025-10-15T16:10:53, 450, 200, 1
1, 2025-10-15T16:10:54, 450, 82, 1
1, 2025-10-15T16:10:55, 450, 200, 1
1, 2025-10-15T16:10:59, 945, 200, 1
1, 2025-10-15T16:11:00, 945, 82, 1
2, 2025-10-15T16:11:16, 55, 82, 1
2, 2025-10-15T16:11:17, 55, 200, 1
2, 2025-10-15T16:11:20, 450, 200, 1
2, 2025-10-15T16:11:21, 450, 82, 1
2, 2025-10-15T16:11:22, 450, 200, 1
2, 2025-10-15T16:11:26, 945, 200, 1
2, 2025-10-15T16:11:27, 945, 82, 1
```

Grade B

Requires grade C. Parts generated in source 2 (type 2) should have a sequence that includes:

1. Moving from Source2 to Process2
2. Running Process2
3. Moving to Process 1
4. Running Process1

5. Moving to Sink

The system should make it possible to run parts of type 1 and type 2 at the same time in the system. It should be possible at any time to add parts in both sources. The system should continue to pick parts until all are completed.

Grade A

Requires grade B. Create an HMI that can let the user control the crane to move to various positions and save the positions to the JSON file used earlier. The file can then be executed, causing the crane to follow the points. The X and Y variables for all positions in the simulation should be displayed in the HMI with the possibility to select one and update its location to the crane's current location.

You should also be able to add the operations vacuum and run process in the sequence from you HMI.

This means that you can create a complete JSON file from your HMI.